

Mohd Shahid

File operations are essential in programming for storing, retrieving, and processing data. Different file types require different handling methods.

1. Text Files (.txt)

Writing to a Text File

```
with open("sample.txt", "w") as file:  
    file.write("Hello, World!")
```

Reading from a Text File

```
with open("sample.txt", "r") as file:  
    content = file.read()  
    print(content)
```

2. CSV Files (.csv)

CSV (Comma-Separated Values) files are commonly used for tabular data.

Writing to a CSV File

```
import csv
```

```
data = [  
    ["Name", "Age"],  
    ["Alice", 25],  
    ["Bob", 30]  
]
```

```
with open("people.csv", "w", newline="") as file:  
    writer = csv.writer(file)  
    writer.writerows(data)
```

Reading from a CSV File

```
import csv
```

```
with open("people.csv", "r") as file:  
    reader = csv.reader(file)  
    for row in reader:  
        print(row)
```

3. JSON Files (.json)

JSON is widely used for storing structured data.

Writing to a JSON File

```
import json
```

```
data = {  
    "name": "Alice",  
    "age": 25  
}
```

```
with open("data.json", "w") as file:  
    json.dump(data, file, indent=4)
```

Reading from a JSON File

```
import json
```

```
with open("data.json", "r") as file:  
    data = json.load(file)  
    print(data)
```

4. Binary Files (.bin)

Binary files store data in binary format.

Writing to a Binary File

```
data = b"Hello Binary"
```

```
with open("sample.bin", "wb") as file:  
    file.write(data)
```

Reading from a Binary File

```
with open("sample.bin", "rb") as file:  
    data = file.read()  
    print(data)
```

5. Excel Files (.xlsx)

Excel files can be handled using the Python library openpyxl.

Writing to an Excel File

```
from openpyxl import Workbook
```

```
wb = Workbook()
```

```
ws = wb.active
```

```
ws["A1"] = "Name"
```

```
ws["B1"] = "Age"
```

```
ws.append(["Alice", 25])
```

```
wb.save("employees.xlsx")
```

Reading from an Excel File

```
from openpyxl import load_workbook
```

```
wb = load_workbook("employees.xlsx")
```

```
ws = wb.active
```

```
for row in ws.iter_rows(values_only=True):  
    print(row)
```

6. XML Files (.xml)

Writing to an XML File

```
import xml.etree.ElementTree as ET
```

```
root = ET.Element("person")
```

```
name = ET.SubElement(root, "name")
```

```
name.text = "Alice"
```

```
tree = ET.ElementTree(root)
```

```
tree.write("person.xml")
```

Reading from an XML File

```
import xml.etree.ElementTree as ET
```

```
tree = ET.parse("person.xml")
```

```
root = tree.getroot()
```

```
print(root.find("name").text)
```

File Modes

Mode Description

r	Read only
w	Write (creates/overwrites file)
a	Append to file
x	Create file; error if exists
rb	Read binary
wb	Write binary
r+	Read and write

Example: Appending Data

```
with open("sample.txt", "a") as file:  
    file.write("\nNew line added.")
```

Best Practices

1. Use `with open()` to automatically close files.
2. Handle exceptions using `try-except`.
3. Choose the correct file mode.
4. Validate file existence before reading.
5. Use appropriate libraries for specialized formats (CSV, JSON, Excel, XML).

Example with Error Handling

```
try:  
    with open("sample.txt", "r") as file:  
        print(file.read())  
except FileNotFoundError:  
    print("File not found.")
```

These techniques cover the most common file operations used in Python applications.

1. Text Files (.txt) – Simple Data Storage

Text files are useful for storing plain text data such as logs, notes, configuration settings, and reports.

Writing to a Text File

```
with open("notes.txt", "w") as file:
    file.write("Python File Handling\n")
    file.write("Learning text files.")
```

Reading a Text File

```
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)
```

Reading Line by Line

```
with open("notes.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Appending Data

```
with open("notes.txt", "a") as file:
    file.write("\nNew information added.")
```

Common Uses

- Log files
 - User notes
 - Configuration files
 - Reports
-

2. CSV Files – Structured Data Storage

CSV (Comma-Separated Values) files store tabular data.

Example CSV:

```
Name,Age,City
Alice,25,Delhi
Bob,30,Mumbai
```

Writing CSV Data

```
import csv
```

```
data = [  
    ["Name", "Age", "City"],  
    ["Alice", 25, "Delhi"],  
    ["Bob", 30, "Mumbai"]  
]
```

```
with open("people.csv", "w", newline="") as file:  
    writer = csv.writer(file)  
    writer.writerows(data)
```

Reading CSV Data

```
import csv
```

```
with open("people.csv", "r") as file:  
    reader = csv.reader(file)
```

```
    for row in reader:  
        print(row)
```

Output:

```
['Name', 'Age', 'City']  
['Alice', '25', 'Delhi']  
['Bob', '30', 'Mumbai']
```

Using Dictionary Reader

```
import csv
```

```
with open("people.csv", "r") as file:  
    reader = csv.DictReader(file)
```

```
    for row in reader:  
        print(row["Name"], row["City"])
```

Output:

```
Alice Delhi  
Bob Mumbai
```

Advantages of CSV

- Easy to read

- Supported by Excel
 - Suitable for structured data
 - Lightweight format
-

3. Error Handling for File Operations

File operations often fail because:

- File does not exist
- Permission denied
- Invalid file path
- Disk issues

Basic Exception Handling

try:

```
with open("data.txt", "r") as file:  
    content = file.read()  
    print(content)
```

except FileNotFoundError:

```
    print("The file does not exist.")
```

Multiple Exceptions

try:

```
with open("data.txt", "r") as file:  
    print(file.read())
```

except FileNotFoundError:

```
    print("File not found.")
```

except PermissionError:

```
    print("Permission denied.")
```

except Exception as e:

```
    print("Unexpected error:", e)
```

Safe File Writing

try:

```
with open("output.txt", "w") as file:
```

```
file.write("Data saved successfully.")
```

```
except IOError as e:  
    print("I/O Error:", e)
```

4. Context Managers (with Statement)

A context manager automatically handles resource cleanup.

Without with:

```
file = open("data.txt", "r")
```

```
try:  
    content = file.read()  
finally:  
    file.close()
```

With with:

```
with open("data.txt", "r") as file:  
    content = file.read()
```

The file closes automatically.

Benefits

Automatic Resource Management

```
with open("log.txt", "w") as file:  
    file.write("Log entry")
```

Cleaner Code

```
with open("input.txt") as source, \  
    open("output.txt", "w") as destination:  
  
    destination.write(source.read())
```

Prevents Resource Leaks

Files are closed even if an exception occurs.

5. Code Organization with Multiple Files and Modules

As projects grow, keeping everything in one file becomes difficult.

Poor Structure

```
# main.py
# Hundreds of lines handling files,
# calculations, reports, and UI
```

Better Structure

```
project/
|
├── main.py
├── file_handler.py
├── csv_manager.py
├── config.py
└── utils.py
```

Example Module

file_handler.py

```
def read_text_file(filename):
    with open(filename, "r") as file:
        return file.read()
```

main.py

```
from file_handler import read_text_file

content = read_text_file("notes.txt")
print(content)
```

Creating Reusable Functions

```
# csv_manager.py
```

```
import csv
```

```
def load_csv(filename):
    with open(filename, "r") as file:
        return list(csv.DictReader(file))
```

Usage:

```
from csv_manager import load_csv
```

```
data = load_csv("people.csv")  
print(data)
```

Package Structure

```
project/  
|  
├── main.py  
|  
└── file_utils/  
    ├── __init__.py  
    ├── text.py  
    └── csv_tools.py
```

Import:

```
from file_utils.text import read_text_file
```

Benefits:

- Easier maintenance
 - Better testing
 - Reusability
 - Improved readability
-

6. Advanced Debugging Techniques for File Operations

Check File Existence

```
import os
```

```
filename = "data.txt"
```

```
if os.path.exists(filename):  
    print("File exists")  
else:  
    print("File not found")
```

Display Full Path

```
import os
```

```
print(os.path.abspath("data.txt"))
```

Useful when files seem "missing".

Logging Instead of Print

```
import logging
```

```
logging.basicConfig(level=logging.INFO)
```

```
logging.info("Opening file")
```

Output:

```
INFO:root:Opening file
```

Debug File Reads

```
with open("data.txt", "r") as file:
```

```
    for line_number, line in enumerate(file, start=1):  
        print(f"{line_number}: {line.strip()}")
```

Trace Exceptions

```
import traceback
```

```
try:
```

```
    with open("missing.txt") as file:  
        print(file.read())
```

```
except Exception:
```

```
    traceback.print_exc()
```

Provides detailed error information.

Use Assertions

```
import os
```

```
filename = "data.txt"
```

```
assert os.path.exists(filename), "File does not exist"
```

Debugging CSV Issues

```
import csv
```

```
with open("people.csv") as file:  
    reader = csv.reader(file)
```

```
    for row in reader:  
        print("Columns:", len(row))  
        print(row)
```

Helps identify malformed rows.

Recommended Project Example

StudentManagement/

|

├— main.py

├— students.csv

|

├— file_handler.py

├— csv_manager.py

├— logger.py

└— reports.py

Responsibilities:

File	Purpose
main.py	Application entry point
file_handler.py	Text file operations
csv_manager.py	CSV processing
reports.py	Report generation

File	Purpose
logger.py	Logging and debugging

This separation makes file handling code easier to maintain, test, debug, and extend in larger Python applications.

We use cookies

Cookies help this site function, measure usage, and support marketing. Manage your cookie preferences anytime. Learn more about our [cookie policy](#).

CODE

A great way to practice all those concepts is to build a small **Personal Productivity Suite** consisting of multiple modules. It includes:

- User data storage in text files
- Diary/notes application
- CSV file processing
- Quiz game with score saving
- Persistent to-do list
- File error handling
- Multi-module project organization

Project Structure

```
productivity_suite/  
|  
└─ main.py
```

- └─ user_manager.py
- └─ diary.py
- └─ todo.py
- └─ quiz.py
- └─ csv_reader.py
- └─ file_utils.py
- |
- └─ users.txt
- └─ diary.txt
- └─ todo.txt
- └─ scores.txt
- └─ questions.csv

1. File Utilities Module

file_utils.py

```
def safe_read(filename):  
    try:  
        with open(filename, "r") as file:  
            return file.read()  
    except FileNotFoundError:  
        return ""  
    except PermissionError:  
        return "Permission denied."
```

```
def safe_append(filename, text):  
    try:  
        with open(filename, "a") as file:  
            file.write(text + "\n")  
    except Exception as e:  
        print("Error:", e)
```

2. Save User Data

user_manager.py

```
from file_utils import safe_append
```

```
def save_user():
    name = input("Enter your name: ")
    age = input("Enter your age: ")

    record = f"Name: {name}, Age: {age}"
    safe_append("users.txt", record)

    print("User saved successfully.")
```

3. Diary Application

diary.py

```
from file_utils import safe_append, safe_read

def add_note():
    note = input("Write your note: ")
    safe_append("diary.txt", note)

def view_notes():
    print("\nDiary Entries")
    print("-----")
    print(safe_read("diary.txt"))
```

4. Persistent To-Do List

todo.py

```
from file_utils import safe_append, safe_read

def add_task():
    task = input("Enter task: ")
    safe_append("todo.txt", task)

def view_tasks():
    print("\nTo-Do List")
    print("-----")
    print(safe_read("todo.txt"))
```

5. Quiz Game with Score Saving

questions.csv

Question,Answer
Capital of France?,Paris
2+2?,4
Largest planet?,Jupiter

quiz.py

```
import csv

def play_quiz():
    score = 0

    try:
        with open("questions.csv", "r") as file:
            reader = csv.DictReader(file)

            for row in reader:
                answer = input(row["Question"] + " ")

                if answer.lower() == row["Answer"].lower():
                    score += 1

            print(f"Final Score: {score}")

            with open("scores.txt", "a") as file:
                file.write(f"Score: {score}\n")

    except FileNotFoundError:
        print("Questions file not found.")
```

6. CSV Processing Example

csv_reader.py

```
import csv

def display_questions():

    try:
```



```
with open("questions.csv", "r") as file:
```

```
    reader = csv.reader(file)
```

```
    for row in reader:
```

```
        print(row)
```

```
except FileNotFoundError:
```

```
    print("CSV file not found.")
```

7. Main Program

main.py

```
from user_manager import save_user
```

```
from diary import add_note, view_notes
```

```
from todo import add_task, view_tasks
```

```
from quiz import play_quiz
```

```
from csv_reader import display_questions
```

```
while True:
```

```
    print("\n===== MENU =====")
```

```
    print("1. Save User")
```

```
    print("2. Add Diary Note")
```

```
    print("3. View Diary")
```

```
    print("4. Add Task")
```

```
    print("5. View Tasks")
```

```
    print("6. Play Quiz")
```

```
    print("7. View CSV Data")
```

```
    print("8. Exit")
```

```
    choice = input("Choose: ")
```

```
    if choice == "1":
```

```
        save_user()
```

```
    elif choice == "2":
```

```
        add_note()
```

```
    elif choice == "3":
```

```
view_notes()

elif choice == "4":
    add_task()

elif choice == "5":
    view_tasks()

elif choice == "6":
    play_quiz()

elif choice == "7":
    display_questions()

elif choice == "8":
    print("Goodbye!")
    break

else:
    print("Invalid choice.")
```

Error Handling Practice

Try intentionally causing these errors and handling them:

Missing File

```
try:
    with open("missing.txt") as file:
        print(file.read())
except FileNotFoundError:
    print("File does not exist.")
```

Permission Error

```
try:
    with open("/restricted/file.txt") as file:
        pass
except PermissionError:
    print("Permission denied.")
```

Invalid CSV Format

```
import csv

try:
    with open("bad.csv") as file:
        reader = csv.reader(file)
        for row in reader:
            print(row[5])
except IndexError:
    print("CSV row has fewer columns than expected.")
```

Skills Practiced

- ✓ Text file reading and writing
- ✓ CSV file processing
- ✓ Context managers (with)
- ✓ Error handling (try-except)
- ✓ Data persistence between runs
- ✓ Modular programming
- ✓ Importing functions from modules
- ✓ Building a menu-driven application

This project combines all the file-handling topics into a single practical application that can be expanded with features like deleting tasks, editing notes, user login, and score leaderboards.